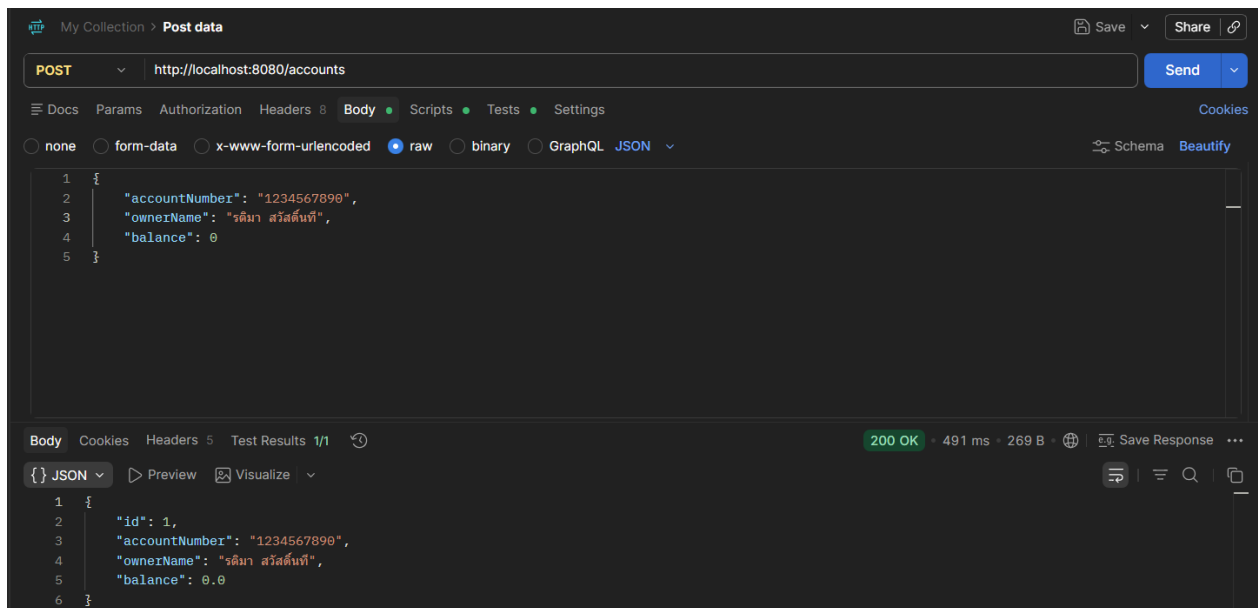


Lab 9: Spring Boot - Transaction

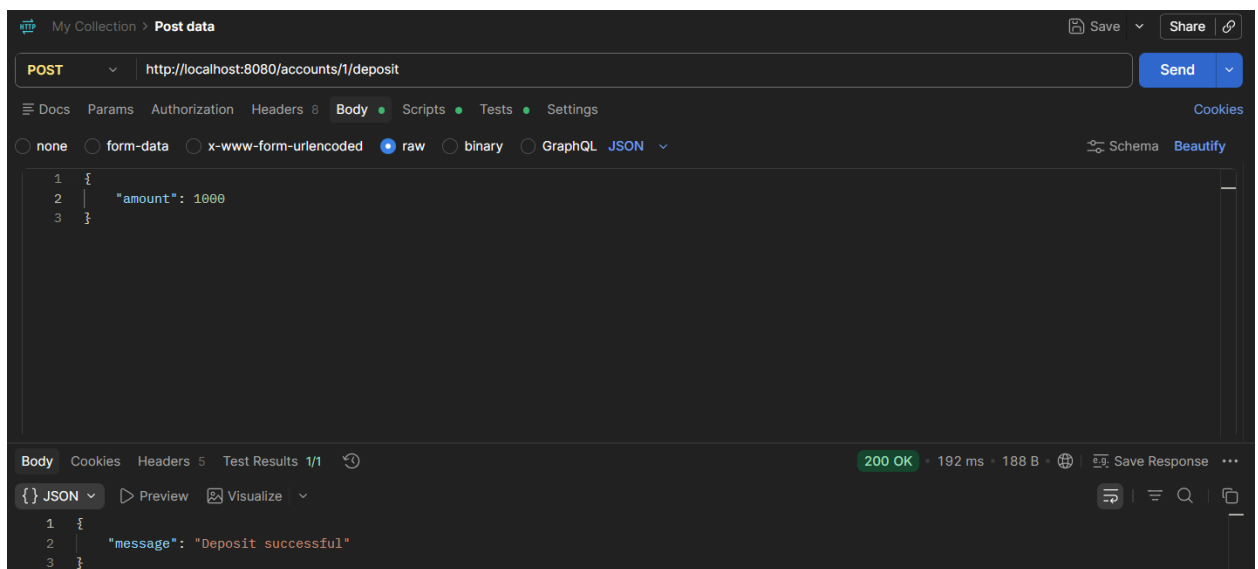
1. สร้าง Account

POST:



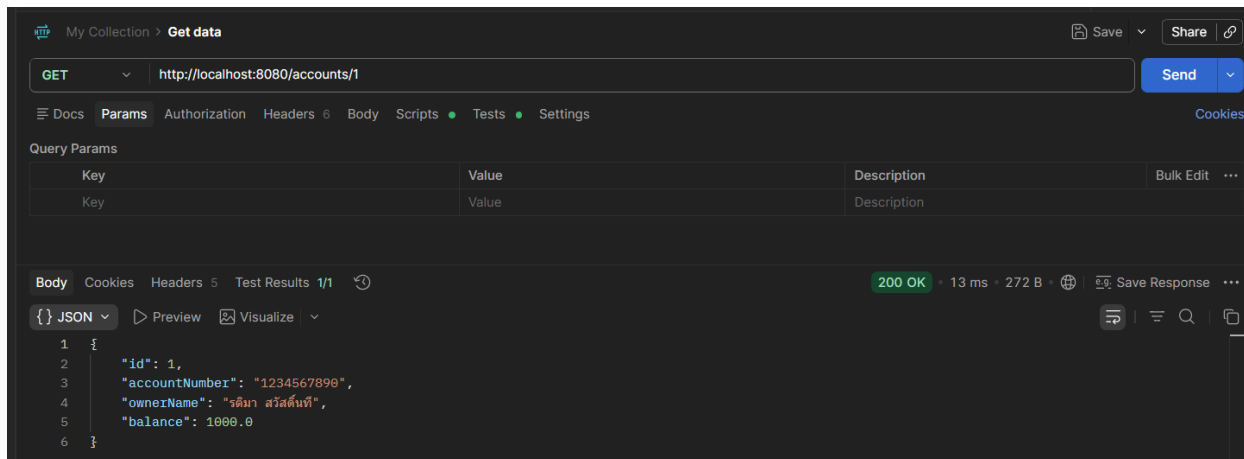
อธิบายผล: ทำการสร้างบัญชีด้วย POST /accounts โดยกำหนด accountNumber,ownerName,balance

2.ฝากเงิน



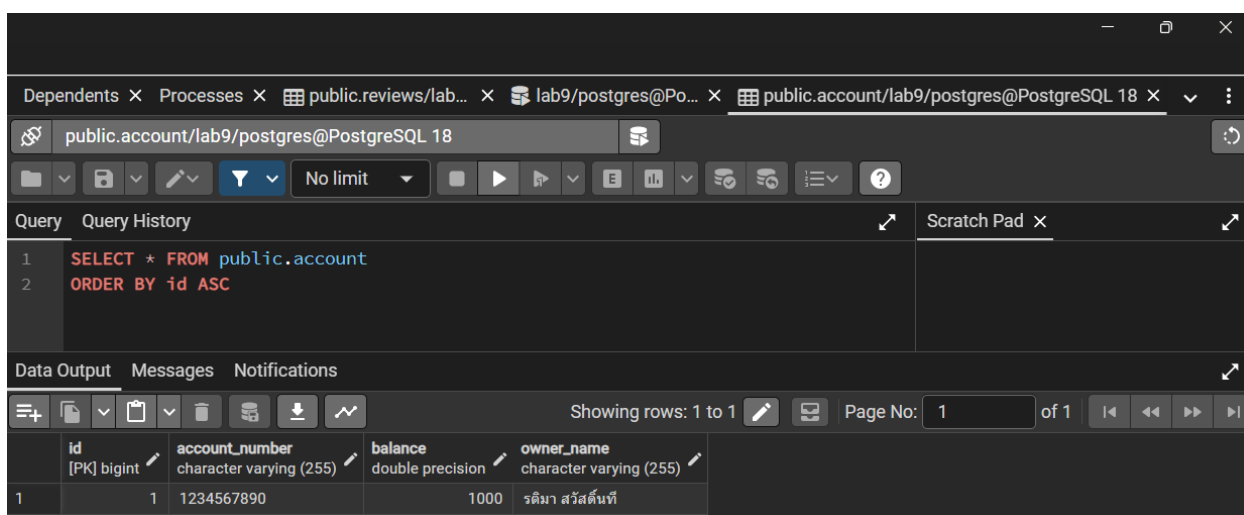
อธิบายผล: ทำการฝากเงิน1000 ผ่าน POST /accounts/1/deposit ตามaccount ที่สร้างไว้ ระบบตอบกลับสถานะ 200 OK และข้อความ Deposit successful แสดงว่าการฝากเงิน

3.ตรวจสอบยอดเงิน



อธิบายผล: ตรวจสอบข้อมูลของ account ผ่าน GET /accounts/1 จะเห็นว่า balance เปลี่ยนเป็น 1,000 บาท

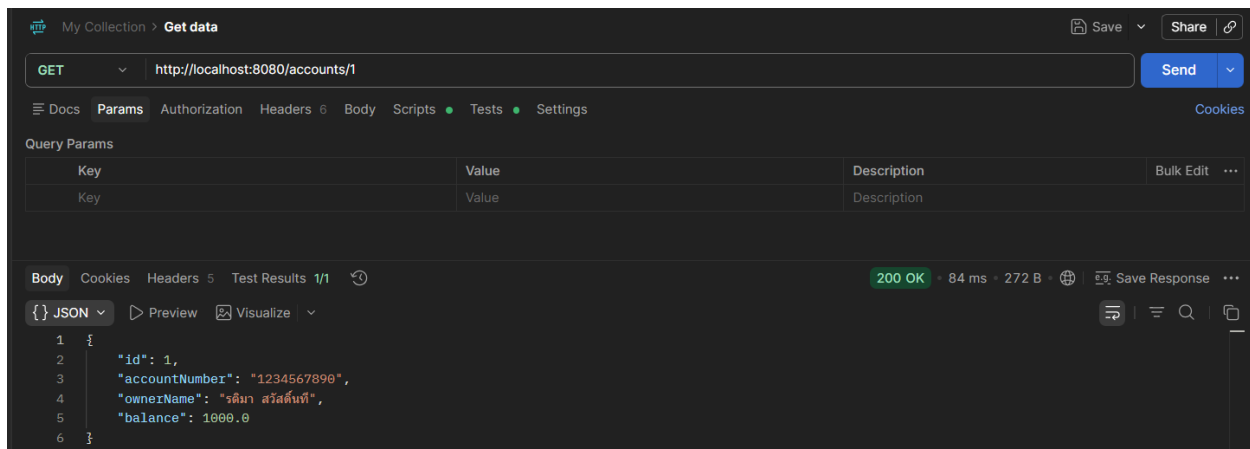
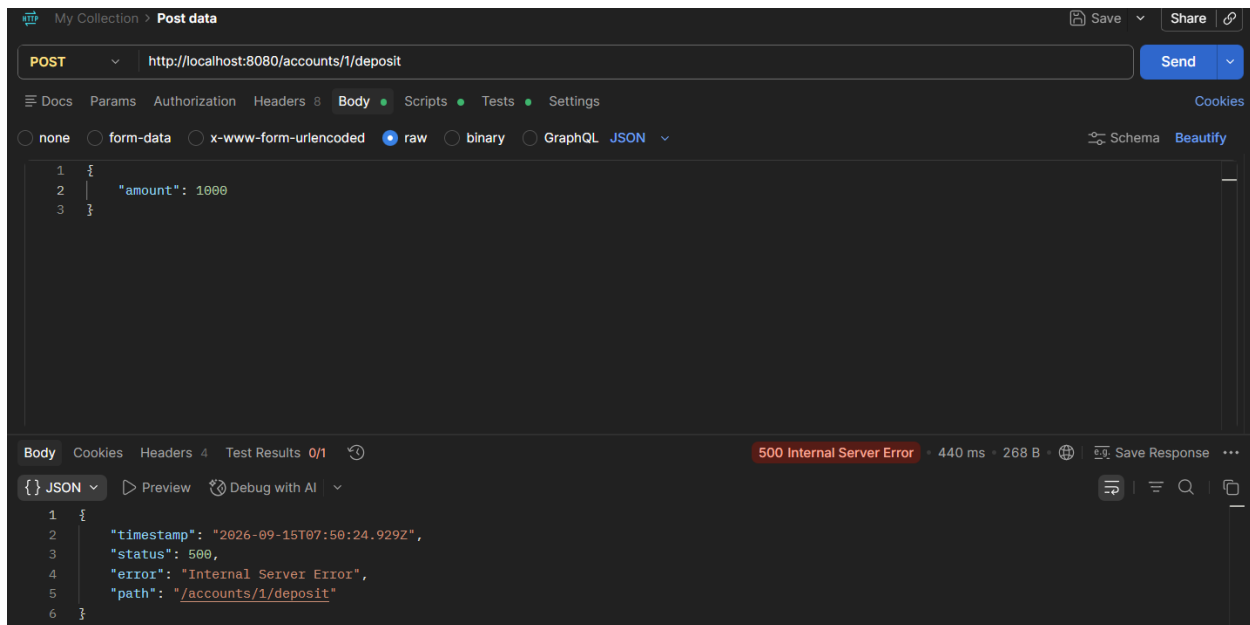
4.ตรวจสอบประวัติการฝากเงิน



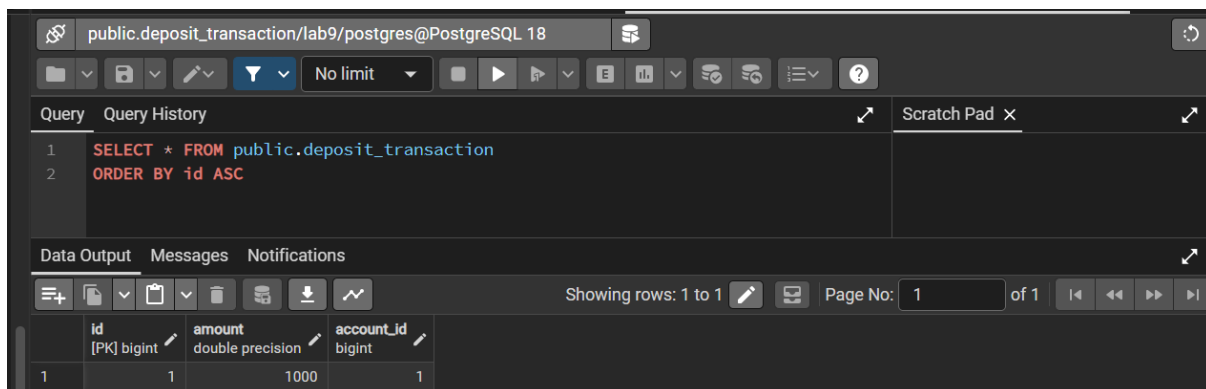
อธิบายผล: ในตาราง account จะเห็นว่าว่ายอดเงินถูกบันทึกไว้เป็น 1,000 บาทตามที่เรฝากไว้

5.ทดลอง Transaction Rollback แบบยังมี @Transactional

Postman:



ตาราง deposit_transaction หลัง Rollback :

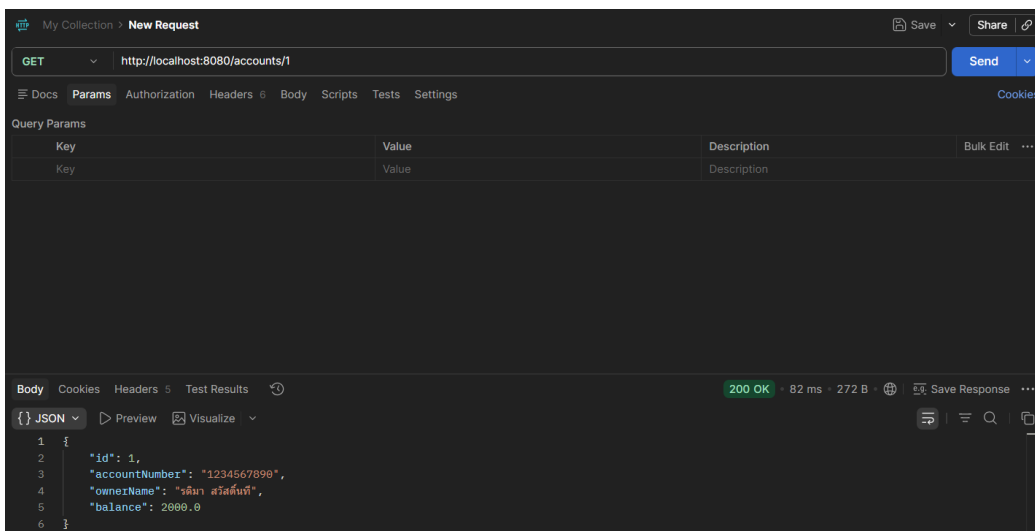
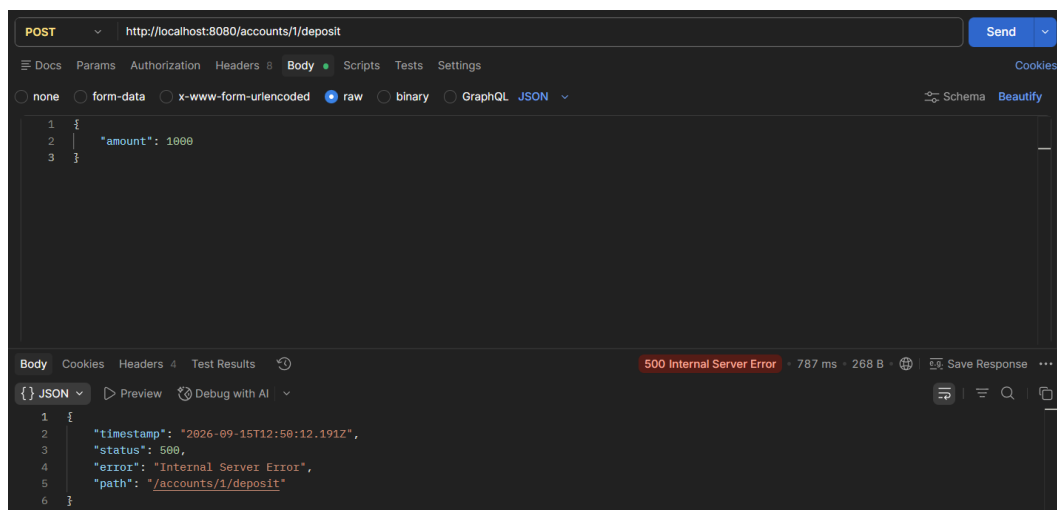


อธิบายผล: เพิ่มคำสั่ง throw new RuntimeException("Test Rollback") ไว้ท้ายเมธอด deposit() แต่อันนี้ยังมี @Transactional อยู่ และทำการฝากเงิน 1000 อีกครั้ง จะเห็นว่าระบบแสดง 500 Internal Server Error และจะเห็นว่า ยอดเงินยัง 1000 เท่าเดิม และในตาราง deposit_transaction ก็ยังไม่มีรายการฝากเงินใหม่เพิ่มขึ้น แสดงว่าการเพิ่มยอดเงินและการบันทึกประวัติการฝากเงินถูกยกเลิก ROLLBACK Transaction ทั้งหมด ทำให้การเปลี่ยนแปลงที่เกิดขึ้นภายในการฝากเงินครั้งนั้นไม่ถูกบันทึกลงฐานข้อมูล

6.ทดลอง Transaction Rollback แบบไม่มี @Transactional

```
public void deposit(Long accountId, Double amount){  
    Account account = accountRepository.findById(accountId).  
  
    account.setBalance(account.getBalance() + amount);  
    accountRepository.save(account);  
}
```

Postman:



```
1 SELECT * FROM public.deposit_transaction
2 ORDER BY id ASC
```

Data Output		Messages	Notifications
	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1
2	3	1000	1

อธิบายผล : Transactional ออกจากเมธอด deposit() แต่มี throw new RuntimeException("Test Rollback") ไว้ แล้วฝากเงิน 1000 postman จะตอบกลับ 500 Internal Server Error แต่ในจะเห็นว่าbalance เป็น 2000 และในdatabase และมีรายการใน deposit_transaction เพิ่มขึ้นเป็น 2 รายการ ผลดังกล่าวแสดงให้เห็นว่าการบันทึกข้อมูลของแต่ละ Repository สามารถเสร็จสิ้นแยกจากกันได้ และ Exception ที่เกิดภายหลังไม่ได้ทำการ Rollback

7.คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ตอบ @Transactional ใช้กำหนดขอบเขตของ Transaction ให้การทำงานหลายขั้นตอนถูกจัดการเป็นหน่วยเดียวกัน หากทำงานสำเร็จจึง Commit แต่ถ้าเกิด RuntimeException ตามกรณีทดลอง Spring จะ Rollback การเปลี่ยนแปลงใน Transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ เพราะการฝากเงินต้องแก้ไขยอด Account และบันทึก DepositTransaction ให้สอดคล้องกัน หากขั้นตอนหนึ่งสำเร็จแต่อีกขั้นตอนล้มเหลว อาจทำให้ข้อมูลในฐานข้อมูลไม่ตรงกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT คือการยืนยันการเปลี่ยนแปลงให้ถูกบันทึกลงฐานข้อมูลอย่างถาวร ส่วน ROLLBACK คือการยกเลิกการเปลี่ยนแปลง ของ Transaction และย้อนกลับไปยังสถานะก่อนเริ่ม Transaction

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ Account 1 บัญชีสามารถมี DepositTransaction ได้หลายรายการ จึงเป็นความสัมพันธ์แบบ One-to-Many โดย DepositTransaction แต่ละรายการอ้างอิง Account ผ่าน account_id

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ ระบบจะทำการ ROLLBACK การเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน Transaction ทำให้ข้อมูลใน Database กลับไปเป็นสถานะก่อนเริ่ม Transaction

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ เมื่อลบ @Transactional ออก แล้วเกิด Error การเปลี่ยนแปลงที่เกิดขึ้นก่อนหน้านี้ ไม่ถูก Rollback ทั้งหมด